



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI-Assisted Development & API Integration

Eng. Fatma Elraey*Lab 2 — Building a Custom AI App (Final Lab)*

Field	Detail
Instructor	Fatma Elraey
Course	AI-Assisted Development & API Integration (2-Day Course)
Session	Day 2 of 2 — Lab, 3 Hours (Final Lab)
Focus	Building a lightweight Gemini-powered chat app or AI-powered content summarizer, with secure API keys, real conversation memory, and clean streaming output.

Before You Start

You'll need Python installed, a Google Gemini API key, and the `google-genai` and `python-dotenv` packages. The same concepts still apply to OpenAI or Anthropic; only the SDK calls differ. Install them with:

```
pip install google-genai python-dotenv --break-system-packages
```

By the end of this lab, you'll have a working command-line AI app that remembers the conversation, streams its replies, and never has a secret hardcoded in the source file.

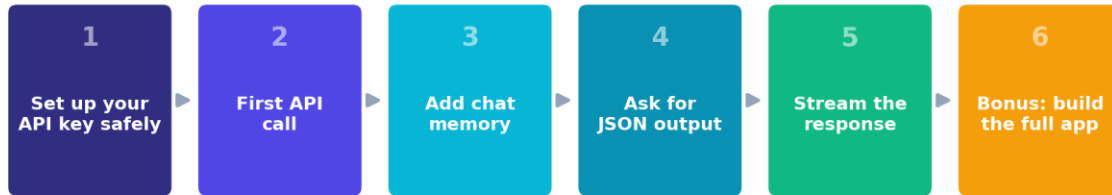


Figure L.1 — Six steps: secure the key, call the API, add memory, get JSON, stream it, then combine it all.

✓ QUICK CHECK — Before You Start

Q1. What are the main message/instruction roles you'll use conceptually in a chat API?

Answer: system instruction, user, and model/assistant.

Q2. Why must conversation history be included in requests or managed by a chat session?

Answer: The API is stateless. Gemini chat sessions manage the conversation history for you across `send_message()` calls, but the model still needs that history as context.

Q3. What Gemini setting requests structured JSON output?

Answer: `response_mime_type="application/json"`



01

Set Up Your API Key Safely

Before writing a single line that calls the API, get the API key handling right — this is the single most common security mistake in beginner AI projects.

- Create a file named `.env` in your project folder containing `GEMINI_API_KEY=your-key-here`.
- Add `.env` to your `.gitignore` file immediately, before you forget.
- Never paste your key directly into a `.py` file, and never commit it to version control.
- Write a tiny script that loads the key and confirms it's readable, without ever printing the full key.

Why this matters: A leaked API key is a real, common, and expensive mistake — someone else can rack up usage on your account within minutes of a key appearing in a public GitHub repo.

Create this file:

```
# .env
GEMINI_API_KEY=your-real-key-here
```

Test script:

```
import os
from dotenv import load_dotenv

load_dotenv()
api_key = os.environ["GEMINI_API_KEY"]
print("Key loaded:", api_key[:4] + "...")
```

02

Your First API Call

Write the smallest possible script that sends a prompt to the model and prints the reply.

- Import the Gemini SDK, create a client, and send a single prompt to the model.
- Print just the reply text, not the whole response object.
- Try changing the prompt and running it again to see different responses.

Why this matters: This exact Gemini pattern — build a `genai.Client()`, call `client.models.generate_content()`, and read `response.text` — is the foundation every exercise after this one builds on.

Try this:



```
from dotenv import load_dotenv
from google import genai
load_dotenv()
client = genai.Client()
response = client.models.generate_content(
    model="gemini-3.7-flash",
    contents="Say hello in one short sentence."
)
print(response.text)
```

03

Add Conversation Memory

Turn your one-off script into something that remembers previous turns by using a Gemini chat session.

- Create a chat session with a system instruction; Gemini keeps the growing chat history for that session.
- Write a function that sends the user's message through the chat session and returns `response.text`.
- Test that it actually remembers: tell it something in one call, then ask about it in the next.

Why this matters: AI chat "memory" still comes from conversation history. With Gemini, the chat session manages that growing history for you, instead of you manually appending every message to a list.

Try this:

```
from google.genai import types

chat_session = client.chats.create(
    model="gemini-3.7-flash",
    config=types.GenerateContentConfig(
        system_instruction="You are a friendly assistant."
    )
)

def ask(user_input):
    response = chat_session.send_message(user_input)
    return response.text
print(ask("My favorite color is blue.))
print(ask("What is my favorite color?"))
```

Expected output:

```
"Nice! Blue is a great color."
"Your favorite color is blue."
```

04

Ask for Structured JSON Output

Build a small summarizer that returns clean, parseable JSON instead of a paragraph of prose.

- Write a function `summarize(text)` that asks the model for a JSON object with a summary string and a `key_points` list.
- Use `response_mime_type="application/json"` in Gemini configuration and parse `response.text` with `json.loads()`.
- Test it on a paragraph of your choice and print the summary and each key point separately.

Why this matters: This is exactly how you'd feed an LLM's output into the rest of a real application — a database, a UI component, another function — none of which want to parse free-form prose.

Try this:

```
import json
from google.genai import types
def summarize(text):
    response = client.models.generate_content(
        model="gemini-3.7-flash",
        contents=text,
        config=types.GenerateContentConfig(
            system_instruction=(
                "Summarize the text. Return JSON with a summary string and a
                key_points array of strings."
            ),
            response_mime_type="application/json"
        )
    )
    return json.loads(response.text)

result = summarize("...your paragraph here...")
print(result["summary"])
for point in result["key_points"]:
    print("-", point)
```

05

Stream the Response

Make replies appear word by word as they're generated, instead of waiting for the entire response to finish.

- Use Gemini's `generate_content_stream()` (or `send_message_stream()` for a chat session).
- Loop over the streamed chunks and print `chunk.text` as each piece arrives, with no line break between pieces.
- Compare how it feels versus waiting for the full (non-streamed) response from Exercise 02.



Why this matters: Streaming is why AI chat replies can appear to "type" in real time instead of showing up all at once after a long pause — for longer replies, it makes an app feel dramatically more responsive.

Try this:

```
stream = client.models.generate_content_stream(  
    model="gemini-3.7-flash",  
    contents="Write a short poem about the ocean."  
)  
  
for chunk in stream:  
    if chunk.text:  
        print(chunk.text, end="", flush=True)  
print()
```



Bonus Stretch Goal — Build the Full App

Combine everything from this lab into one small, complete command-line chat application: secure API key loading, conversation memory, and streaming output, all together.

- Load the API key from `.env` using `python-dotenv` (Exercise 01).
- Keep a Gemini chat session across turns so the session manages conversation history (Exercise 03).
- Stream each reply with `chat_session.send_message_stream()` (Exercise 05); the chat session keeps the conversation context while you display chunks as they arrive.
- Wrap it in a loop that keeps asking for input until the user types 'quit'.
- Optional: adapt this into a content summarizer instead — read a block of pasted text, stream a structured summary back.

Why this matters: This small app already has the three pillars of almost every production AI chat product: secure credentials, managed conversation state, and a responsive streaming experience.

Starter template:

```
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
client = genai.Client()
chat_session = client.chats.create(
    model="gemini-3.7-flash",
    config=types.GenerateContentConfig(
        system_instruction="You are a helpful assistant."
    )
)

def send(user_input):
    stream = chat_session.send_message_stream(user_input)
    full_reply = ""
    for chunk in stream:
        if chunk.text:
            print(chunk.text, end="", flush=True)
            full_reply += chunk.text
    print()
    return full_reply
while True:
    user_input = input("You: ")
    if user_input.lower() == "quit":
        break
    send(user_input)
```

Congratulations — you've gone from using AI tools to building a real, working AI application of your own.